

Northeastern University
Class Assignment 1: EECE 7205
Professor: Dr. Fatema Nafa

Rohini Bidnurmah

NUID: 003916201

September 2026

Contents

1	Problem 1: Bubble Sort	3
1.1	Problem Statement	3
1.2	Description of Approach	3
1.2.1	Algorithm Chosen and Correctness Explanation	3
1.2.2	Pseudocode	4
1.2.3	Time and Space Complexity Derivation	4
1.3	Answers to the Assignment Questions	5
1.4	Test Cases and Screenshots	5
2	Problem 2: Insertion Sort	6
2.1	Problem Statement	6
2.2	Description of Approach	6
2.2.1	Algorithm Chosen and Correctness Explanation	6
2.2.2	Pseudocode	7
2.2.3	Time and Space Complexity Derivation	7
2.3	Answers to the Assignment Questions	8
2.4	Test Cases and Screenshots	8
3	Problem 3: Selection Sort	9
3.1	Problem Statement	9
3.2	Description of Approach	9
3.2.1	Algorithm Chosen and Correctness Explanation	9
3.2.2	Pseudocode	10
3.2.3	Time and Space Complexity Derivation	10
3.3	Answers to the Assignment Questions	10
3.4	Test Cases and Screenshots	11
4	Problem 4: Quick Sort	12
4.1	Problem Statement	12
4.2	Description of Approach	12
4.2.1	Algorithm Chosen and Correctness Explanation	12
4.2.2	Pseudocode	13
4.2.3	Time and Space Complexity Derivation	14
4.3	Answers to the Assignment Questions	15
4.4	Test Cases and Screenshots	15
5	Problem 5: Merge Sort	16
5.1	Problem Statement	16
5.2	Description of Approach	16
5.2.1	Algorithm Chosen and Correctness Explanation	16
5.2.2	Pseudocode	17
5.2.3	Time and Space Complexity Derivation	18
5.3	Answers to the Assignment Questions	19
5.4	Test Cases and Screenshots	20
6	Problem 6: Heap Sort	22
6.1	Problem Statement	22
6.2	Description of Approach	22
6.2.1	Algorithm Chosen and Correctness Explanation	22
6.2.2	Pseudocode	23
6.2.3	Time and Space Complexity Derivation	23
6.3	Answers to the Assignment Questions	24
6.4	Test Cases and Screenshots	25

7	Problem 7: Comparing the Six Algorithms	26
7.1	Problem Statement	26
7.2	Description of Approach	27
7.2.1	Algorithm Chosen and Correctness Explanation	27
7.2.2	Pseudocode	27
7.2.3	Time and Space Complexity Derivation	27
7.3	Empirical Results and Analysis	28
7.3.1	1. Comparison Counts ($n = 8$)	28
7.3.2	2. Execution Time Benchmarks (in milliseconds)	28
7.3.3	3. Correctness Check	29
7.3.4	4. Theoretical vs. Empirical Growth Evaluation	29
7.3.5	5. Algorithm Selection Justification	29
7.4	Test Cases and Screenshots	30

1 Problem 1: Bubble Sort

Description: Bubble Sort is a fundamental comparison-based sorting technique that repeatedly traverses an array, compares adjacent elements, and swaps them if they are in the wrong order. Higher values "bubble up" to the end of the array with each pass.

Question: Implement an optimized Bubble Sort algorithm that sorts an array of N integers in ascending order and terminates early if the array becomes sorted before completing $N - 1$ passes.

1.1 Problem Statement

Input: A 1D array `arr` of size N containing unsorted integers.

Task: Iterate through the array using nested loops. Compare adjacent elements `arr[j]` and `arr[j + 1]`. Swap them if `arr[j] > arr[j + 1]`. Utilize an early-stopping boolean flag to abort execution if no swaps occur during a full pass. Count the comparisons and swaps, and print the array after every pass.

Output: The mutated array `arr` sorted in non-decreasing order, plus the comparison and swap totals.

Constraints: $N \geq 0$; array elements are 32-bit signed integers.

Edge Cases:

- $N = 0$ or $N = 1$: Already sorted array (returns immediately).
- Fully sorted input (e.g., `[1, 2, 3, 4, 5]`) $\implies$ early termination after pass 1).
- Reverse-sorted input (e.g., `[5, 4, 3, 2, 1]`) $\implies$ triggers maximum swaps).
- Arrays with duplicate or identical elements (e.g., `[3, 3, 3, 3]`) $\implies$ no swaps performed).

1.2 Description of Approach

1.2.1 Algorithm Chosen and Correctness Explanation

The approach uses an optimized version of the standard Bubble Sort:

1. **Pass Loop:** An outer loop runs from $i = 0$ to $N - 2$. Each pass guarantees that the largest unsorted element settles into its correct final position at index $N - 1 - i$.
2. **Comparison Loop:** An inner loop traverses the unsorted prefix from $j = 0$ to $N - 2 - i$. If `arr[j] > arr[j + 1]`, the two items are swapped.
3. **Early Termination Flag:** A boolean flag `swapped` is initialized to `false` before each inner loop pass. If a swap occurs, `swapped` is set to `true`. If the inner loop finishes with `swapped == false`, no elements were out of order, meaning the array is fully sorted, and execution terminates.

Correctness Justification: By induction, after i passes, the last i elements are guaranteed to be the largest elements in sorted order. Furthermore, if a full pass over the remaining unsorted section yields zero swaps, then `arr[j] ≤ arr[j + 1]` holds for all adjacent indices, proving global monotonic non-decreasing order.

1.2.2 Pseudocode

Algorithm 1 Bubble Sort with Early Termination, Counters and Pass Printing

```

1: procedure BUBBLESORT(arr, n, ref comparisons, ref swaps)
2:   for i  $\leftarrow$  0 to n - 2 do
3:     swapped  $\leftarrow$  false
4:     for j  $\leftarrow$  0 to n - i - 2 do
5:       comparisons  $\leftarrow$  comparisons + 1
6:       if arr[j] > arr[j + 1] then
7:         SWAP(arr[j], arr[j + 1])
8:         swaps  $\leftarrow$  swaps + 1
9:         swapped  $\leftarrow$  true
10:      end if
11:    end for
12:    print "Pass ", i + 1
13:    PRINTARRAY(arr, n)
14:    if not swapped then
15:      break
16:    end if
17:  end for
18: end procedure

```

1.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements in the input array.

1. **Worst-Case Inner Comparisons:** In the worst-case (reverse-sorted array), the outer loop executes $N - 1$ times. The inner loop comparisons sum up as:

$$T_{\text{comp}}(N) = \sum_{i=0}^{N-2} (N - 1 - i) = (N - 1) + (N - 2) + \dots + 1 = \frac{N(N - 1)}{2}$$

2. **Worst-Case Swaps:** Each comparison in a reverse-sorted array results in a swap, adding $3 \times \frac{N(N-1)}{2}$ assignment operations.
3. **Best-Case Scenario:** If the array is already sorted, the inner loop performs $N - 1$ comparisons on pass $i = 0$, detects `swapped == false`, and breaks immediately. Total cost: $c \cdot (N - 1)$.

Total Exact Cost Function $T(N)$:

$$T_{\text{worst}}(N) = c_1 \frac{N(N - 1)}{2} + c_2 \frac{N(N - 1)}{2} + c_3 N = \left(\frac{c_1 + c_2}{2} \right) N^2 + \left(c_3 - \frac{c_1 + c_2}{2} \right) N$$

$$T_{\text{best}}(N) = c_1(N - 1) + c_3 = c_1 N + (c_3 - c_1)$$

- **Worst-Case Time Complexity** — $O(N^2)$: Occurs when input is reverse-sorted; requires $\frac{N(N-1)}{2}$ comparisons and swaps.
- **Best-Case Time Complexity** — $O(N)$: Occurs when input is pre-sorted; terminates after 1 outer pass of $N - 1$ comparisons.
- **Average-Case Time Complexity** — $O(N^2)$: On average, expects $\frac{N(N-1)}{4}$ swaps and $\frac{N(N-1)}{2}$ comparisons.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(1)$: Bubble Sort sorts in-place using a few scalar variables (`i`, `j`, `swapped`, `temp`).
- **Total Space Complexity**: $S(N) = O(1)$ auxiliary space.

1.3 Answers to the Assignment Questions

Results on A : 5 passes (the fifth makes no swaps and triggers the early exit), 25 comparisons, 15 swaps.

Run on an already sorted copy of A : The program makes **1 pass**, **7 comparisons** and **0 swaps**. During the first pass no adjacent pair is out of order, so `swapped` stays `false` and the loop breaks. The cost is $N - 1 = 7$ comparisons, which is the $O(N)$ best case, compared with 25 comparisons and 15 swaps on the unsorted A .

1.4 Test Cases and Screenshots

```
PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob1class.cpp -o prob1class } ; if ($?) { .\prob1class }
Original: 34 7 23 32 5 62 14 19
Pass 1: 7 23 32 5 34 14 19 62
Pass 2: 7 23 5 32 14 19 34 62
Pass 3: 7 5 23 14 19 32 34 62
Pass 4: 5 7 14 19 23 32 34 62
Pass 5: 5 7 14 19 23 32 34 62
Sorted: 5 7 14 19 23 32 34 62
Comparisons: 25 Swaps: 15

already sorted copy
original: 5 7 14 19 23 32 34 62
Pass 1: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 7 swaps: 0
PS D:\c++ dsa stuff> 
```

Figure 1: Test Case for Bubble Sort (on A and on the already sorted copy)

2 Problem 2: Insertion Sort

Description: Insertion Sort builds the sorted array incrementally by iteratively extracting an element from the unsorted section and inserting it into its correct position within the sorted prefix.

Question: Implement Insertion Sort to organize an array of N integers in non-decreasing order using shifting operations.

2.1 Problem Statement

Input: A 1D array `arr` of size N .

Task: Maintain a sorted subarray from index 0 to $i - 1$. For each element `key` at index i (from 1 to $N - 1$), shift all elements in the sorted subarray that are greater than `key` one position to the right, then place `key` in the vacated slot.

Output: The mutated array `arr` sorted in ascending order, plus the comparison and shift totals.

Constraints: $N \geq 0$; 32-bit signed integers.

Edge Cases:

- $N \leq 1$: Trivial base case.
- Pre-sorted array: Zero shifts required ($O(N)$ runtime).
- Reverse-sorted array: Maximum shifts required at every index ($O(N^2)$ runtime).
- Array with uniform values: Zero shifts performed.

2.2 Description of Approach

2.2.1 Algorithm Chosen and Correctness Explanation

The algorithm divides the array conceptually into a "sorted" left part and an "unsorted" right part:

1. **Outer Loop:** Iterates index i from 1 to $N - 1$, setting `key` = `arr`[i].
2. **Shifting Inner Loop:** Initializes pointer $j = i - 1$. While $j \geq 0$ and `arr`[j] > `key`, execution shifts `arr`[j] rightward to index $j + 1$ and decrements j .
3. **Insertion:** Once the inner loop completes, the correct slot for `key` is index $j + 1$.

Correctness Justification: Loop Invariant: At the start of iteration i , the subarray `arr`[0... $i - 1$] contains the original elements of that index range in sorted order. Shifting elements strictly greater than `key` maintains monotonic order when `key` is written to index $j + 1$, extending the invariant to `arr`[0... i].

2.2.2 Pseudocode

Algorithm 2 Insertion Sort with Comparison and Shift Counters

```

1: procedure INSERTIONSORT(arr, n, ref comparisons, ref shifts)
2:   for i = 1 to n - 1 do
3:     key ← arr[i]
4:     j ← i - 1
5:     while j ≥ 0 and (comparisons ← comparisons + 1, arr[j] > key) do
6:       arr[j + 1] ← arr[j]
7:       shifts ← shifts + 1
8:       j ← j - 1
9:     end while
10:    arr[j + 1] ← key
11:    print "i = ", i, ", key = ", key
12:    PRINTARRAY(arr, n)
13:  end for
14: end procedure

```

2.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements.

1. **Worst-Case Operations:** In a reverse-sorted array, for each index i , the inner loop shifts i times:

$$T_{\text{worst}}(N) = \sum_{i=1}^{N-1} i = \frac{(N-1)N}{2} = \frac{N^2 - N}{2}$$

2. **Best-Case Operations:** If the array is already sorted, $\text{arr}[j] > \text{key}$ evaluates to **false** on the very first check for every i . The inner loop body executes 0 times.

$$T_{\text{best}}(N) = \sum_{i=1}^{N-1} 1 = N - 1$$

Total Exact Cost Function $T(N)$:

$$T_{\text{worst}}(N) = c_1 \frac{N^2 - N}{2} + c_2 N = \frac{c_1}{2} N^2 + \left(c_2 - \frac{c_1}{2}\right) N$$

$$T_{\text{best}}(N) = c_2(N - 1) = c_2 N - c_2$$

- **Worst-Case Time Complexity** — $O(N^2)$: Quadratic running time due to reverse-ordered element shifts.
- **Best-Case Time Complexity** — $O(N)$: Linear time performance when inputs are pre-sorted or nearly sorted.
- **Average-Case Time Complexity** — $O(N^2)$: Expects $i/2$ comparisons/shifts per element, yielding $\frac{N^2}{4}$ total operations.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(1)$: In-place sorting algorithm utilizing $O(1)$ extra space for **key**, **i**, and **j**.

2.3 Answers to the Assignment Questions

Results on A : 20 comparisons and 15 shifts.

Fewest and most shifts for the same eight numbers:

Ordering	Input	Comparisons	Shifts
Sorted (fewest shifts)	5 7 14 19 23 32 34 62	7	0
Original A	34 7 23 32 5 62 14 19	20	15
Reversed (most shifts)	62 34 32 23 19 14 7 5	28	28

Each shift removes exactly one inversion (a pair that is out of order), so the number of shifts equals the number of inversions in the input. A sorted array has 0 inversions, so nothing is shifted and each key needs a single comparison ($N - 1 = 7$ in total). A reversed array has the maximum $\frac{N(N-1)}{2} = 28$ inversions, so every key moves all the way to the front. Array A has 15 inversions, which matches its 15 shifts.

2.4 Test Cases and Screenshots

```
PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob2class.cpp -o prob2class } ; if ($?) { .\prob2class }
original: 34 7 23 32 5 62 14 19
i = 1, key = 7: 7 34 23 32 5 62 14 19
i = 2, key = 23: 7 23 34 32 5 62 14 19
i = 3, key = 32: 7 23 32 34 5 62 14 19
i = 4, key = 5: 5 7 23 32 34 62 14 19
i = 5, key = 62: 5 7 23 32 34 62 14 19
i = 6, key = 14: 5 7 14 23 32 34 62 19
i = 7, key = 19: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 20 Shifts: 15

best case (sorted ordering)
original: 5 7 14 19 23 32 34 62
i = 1, key = 7: 5 7 14 19 23 32 34 62
i = 2, key = 14: 5 7 14 19 23 32 34 62
i = 3, key = 19: 5 7 14 19 23 32 34 62
i = 4, key = 23: 5 7 14 19 23 32 34 62
i = 5, key = 32: 5 7 14 19 23 32 34 62
i = 6, key = 34: 5 7 14 19 23 32 34 62
i = 7, key = 62: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 7 Shifts: 0

worst case (reversed ordering)
original: 62 34 32 23 19 14 7 5
i = 1, key = 34: 34 62 32 23 19 14 7 5
i = 2, key = 32: 32 34 62 23 19 14 7 5
i = 3, key = 23: 23 32 34 62 19 14 7 5
i = 4, key = 19: 19 23 32 34 62 14 7 5
i = 5, key = 14: 14 19 23 32 34 62 7 5
i = 6, key = 7: 7 14 19 23 32 34 62 5
i = 7, key = 5: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 28 shifts: 28
```

Figure 2: Test Case for Insertion Sort (on A , the best case and the worst case)

3 Problem 3: Selection Sort

Description: Selection Sort divides the input array into a sorted and unsorted region. It repeatedly scans the unsorted region to locate the minimum element and swaps it with the leftmost unsorted element.

Question: Implement Selection Sort to arrange an array of N integers in non-decreasing order.

3.1 Problem Statement

Input: A 1D array `arr` of size N .

Task: For each position i from 0 to $N - 2$, find the index `minidx` of the minimum value in the subarray `arr[i...N - 1]`. Swap `arr[i]` with `arr[minidx]` only when `minidx` $\neq i$.

Output: The mutated array `arr` sorted in ascending order, plus the comparison and swap totals.

Constraints: $N \geq 0$; signed integers.

Edge Cases:

- $N \leq 1$: No operations required.
- Already sorted input: Still executes $\frac{N(N-1)}{2}$ comparisons, but performs zero swaps because every minimum is already in place.
- Reverse-sorted input: Requires $\frac{N(N-1)}{2}$ comparisons and $\lfloor N/2 \rfloor$ swaps (at most $N - 1$ swaps for any input).

3.2 Description of Approach

3.2.1 Algorithm Chosen and Correctness Explanation

The approach prioritizes minimizing the total number of element swaps:

1. **Outer Loop:** Iterates pointer i from 0 to $N - 2$.
2. **Minimum Search:** Sets `minidx` = i and scans j from $i + 1$ to $N - 1$. If `arr[j]` < `arr[minidx]`, updates `minidx` = j .
3. **Swap Step:** After completing the inner loop scan, swaps `arr[i]` with `arr[minidx]` *only if* `minidx` $\neq i$, and counts the swap.

Correctness Justification: By loop invariant, after step i , the prefix `arr[0...i]` contains the $i + 1$ smallest elements of the entire array in fully sorted order. When i reaches $N - 2$, the remaining element at index $N - 1$ is guaranteed to be the maximum element.

3.2.2 Pseudocode

Algorithm 3 Selection Sort with Comparison and Swap Counters

```

1: procedure SELECTIONSORT(arr, n, ref comparisons, ref swaps)
2:   for i = 0 to n - 2 do
3:     minidx  $\leftarrow$  i
4:     for j = i + 1 to n - 1 do
5:       comparisons  $\leftarrow$  comparisons + 1
6:       if arr[j] < arr[minidx] then
7:         minidx  $\leftarrow$  j
8:       end if
9:     end for
10:    if minidx  $\neq$  i then
11:      SWAP(arr[i], arr[minidx])
12:      swaps  $\leftarrow$  swaps + 1
13:    end if
14:    print "Pass ", i + 1, ": min = ", arr[i], " at index ", minidx
15:    PRINTARRAY(arr, n)
16:  end for
17: end procedure

```

3.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of input elements.

1. **Comparison Count:** Independent of initial element ordering, the inner loop comparison count is deterministic:

$$T_{\text{comp}}(N) = \sum_{i=0}^{N-2} (N-1-i) = \frac{N(N-1)}{2}$$

2. **Swap Count:** The outer loop performs at most $N-1$ swaps across all cases.

Total Exact Cost Function $T(N)$:

$$T(N) \leq c_1 \frac{N(N-1)}{2} + c_2(N-1) = \frac{c_1}{2} N^2 + \left(c_2 - \frac{c_1}{2}\right) N - c_2$$

- **Worst-Case Time Complexity** — $O(N^2)$: Always performs $\frac{N(N-1)}{2}$ comparisons.
- **Best-Case Time Complexity** — $O(N^2)$: Performs identical comparison bounds even on pre-sorted arrays.
- **Average-Case Time Complexity** — $O(N^2)$: Quadratic comparison performance regardless of permutation.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(1)$: Operates strictly in-place with constant memory overhead.

3.3 Answers to the Assignment Questions

Results on A: 28 comparisons and **5 swaps**. Passes 2 and 7 find the minimum already in position ($\text{minidx} = i$), so no swap is made.

Selection Sort vs. Bubble Sort: Selection Sort makes **5 swaps** and Bubble Sort makes **15**, so Selection Sort makes fewer. Bubble Sort only swaps adjacent elements, and each such swap removes exactly one inversion. A has 15 inversions, so Bubble Sort needs 15 swaps. Selection Sort instead moves each minimum directly to its final position with a single swap, so it makes at most one swap per pass (at most $N-1 = 7$), and fewer when the minimum is already in place.

3.4 Test Cases and Screenshots

```
PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob3class.cpp -o prob3class } ; if ($?) { .\prob3class }
original: 34 7 23 32 5 62 14 19
Pass 1: min = 5 at index 4 -> 5 7 23 32 34 62 14 19
Pass 2: min = 7 at index 1 -> 5 7 23 32 34 62 14 19
Pass 3: min = 14 at index 6 -> 5 7 14 32 34 62 23 19
Pass 4: min = 19 at index 7 -> 5 7 14 19 34 62 23 32
Pass 5: min = 23 at index 6 -> 5 7 14 19 23 62 34 32
Pass 6: min = 32 at index 7 -> 5 7 14 19 23 32 34 62
Pass 7: min = 34 at index 6 -> 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 28 swaps: 5
PS D:\c++ dsa stuff>
```

Figure 3: Test Case for Selection Sort

4 Problem 4: Quick Sort

Description: Quick Sort is a divide-and-conquer algorithm that selects a "pivot" element, partitions the array such that smaller elements lie to the left of the pivot and larger elements to the right, and recursively sorts the sub-arrays.

Question: Implement Quick Sort using Lomuto's partitioning scheme with explicit element boundary tracking.

4.1 Problem Statement

Input: A 1D array `arr`, a left boundary index `low`, and a right boundary index `high`.

Task: Choose pivot $P = \text{arr}[\text{high}]$. Reorder the array segment `arr[low...high]` into two parts around index p such that $\forall k \in [\text{low}, p - 1], \text{arr}[k] \leq P$ and $\forall k \in [p + 1, \text{high}], \text{arr}[k] > P$. Recursively invoke Quick Sort on `[low, p - 1]` and `[p + 1, high]`. Print the pivot and the subarray for every call, indented by the recursion depth, and print i, j and the array after every swap of the first partition.

Output: The mutated sub-array sorted in ascending order, plus the comparison total.

Constraints: $0 \leq \text{low} \leq \text{high} < N$.

Edge Cases:

- $\text{low} \geq \text{high}$: Base recursion termination condition.
- Already sorted array with fixed end pivot: Leads to highly unbalanced partitions ($O(N^2)$ worst case).
- Array with all equal elements: Lomuto partition places all elements into one partition.

4.2 Description of Approach

4.2.1 Algorithm Chosen and Correctness Explanation

The approach uses Lomuto's Partitioning strategy:

1. **Pivot Choice:** Select the end element $\text{pivot} = \text{arr}[\text{high}]$.
2. **Boundary Tracking:** Maintain $\text{swapmarker} = \text{low} - 1$ (the index i). Loop currindex (the index j) from low to $\text{high} - 1$.
3. **Partitioning Logic:** If $\text{arr}[\text{currindex}] \leq \text{pivot}$, increment swapmarker and swap $\text{arr}[\text{swapmarker}]$ with $\text{arr}[\text{currindex}]$ (skipped when both indices are equal).
4. **Pivot Placement:** Finally, swap $\text{arr}[\text{swapmarker} + 1]$ with $\text{arr}[\text{high}]$ and return the pivot index $p = \text{swapmarker} + 1$.

Correctness Justification: During loop execution, elements in range `[low...swapmarker]` are guaranteed to be $\leq \text{pivot}$, while elements in range `[swapmarker + 1...currindex - 1]` are $> \text{pivot}$. Swapping the pivot to index $\text{swapmarker} + 1$ places it precisely between the two partitions.

4.2.2 Pseudocode

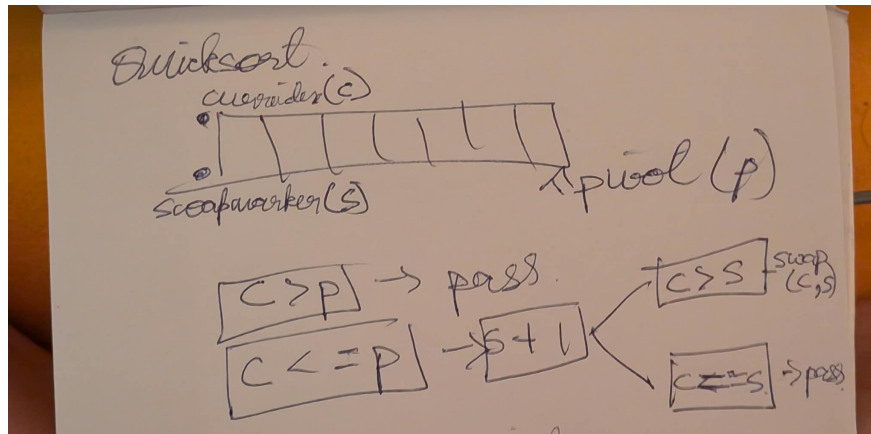


Figure 4: Flowchart illustrating the Quick Sort logic with last element as pivot.

Algorithm 4 Quick Sort (Lomuto Partitioning) with Depth Tracking and Print Output

```

1: procedure PARTITION(arr, low, high, ref comparisons, depth)
2:   pivot  $\leftarrow$  arr[high]
3:   swapmarker  $\leftarrow$  low - 1
4:   for currindex = low to high - 1 do
5:     comparisons  $\leftarrow$  comparisons + 1
6:     if arr[currindex] > pivot then
7:       continue
8:     end if
9:     if arr[currindex]  $\leq$  pivot then
10:      swapmarker  $\leftarrow$  swapmarker + 1
11:      if swapmarker == currindex then
12:        continue
13:      end if
14:      if currindex > swapmarker then
15:        SWAP(arr[swapmarker], arr[currindex])
16:        if depth == 0 then
17:          print " i = ", swapmarker, ", j = ", currindex
18:          PRINTARRAY(arr + low, high - low + 1)
19:        end if
20:      end if
21:    end if
22:  end for
23:  SWAP(arr[swapmarker + 1], arr[high])
24:  if depth == 0 then
25:    print " pivot swapped into place, i + 1 = ", swapmarker + 1
26:    PRINTARRAY(arr + low, high - low + 1)
27:    print " final position of pivot: index ", swapmarker + 1
28:  end if
29:  return swapmarker + 1
30: end procedure

31: procedure QUICKSORT(arr, low, high, ref comparisons, depth)
32:  if low < high then
33:    print 2 · depth spaces, "pivot = ", arr[high]
34:    PRINTARRAY(arr + low, high - low + 1)
35:    p  $\leftarrow$  PARTITION(arr, low, high, comparisons, depth)
36:    QUICKSORT(arr, low, p - 1, comparisons, depth + 1)
37:    QUICKSORT(arr, p + 1, high, comparisons, depth + 1)
38:  end if
39: end procedure

```

4.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let $N = \text{high} - \text{low} + 1$.

1. **Best/Average Case Recurrence:** When partitions split evenly into halves of size $N/2$:

$$T(N) = 2T(N/2) + O(N) \implies T(N) = O(N \log N) \quad (\text{via Master Theorem})$$

2. **Worst-Case Recurrence:** When partitions split into 0 and $N - 1$ elements (e.g., sorted array with last element pivot):

$$T(N) = T(N - 1) + T(0) + cN = c \sum_{k=1}^N k = c \frac{N(N + 1)}{2} = O(N^2)$$

- **Worst-Case Time Complexity** — $O(N^2)$: Occurs under extreme unbalanced partition splits.
- **Best-Case Time Complexity** — $O(N \log N)$: Occurs under perfectly balanced mid-point splits.
- **Average-Case Time Complexity** — $O(N \log N)$: Expected performance across random permutations.

Space Complexity Derivation:

- **Auxiliary Stack Memory** — $O(\log N)$ **Average**, $O(N)$ **Worst Case**: Caused by recursive call stack depth.

4.3 Answers to the Assignment Questions

Results on A: 14 comparisons. The first partition uses pivot 19 and places it at index 3, giving [7, 5, 14, 19, 34, 62, 23, 32].

Run on an already sorted copy of A: 28 comparisons, which is $\frac{N(N-1)}{2}$. The last element is the largest, so the pivot ends at the last index and the split is $N - 1$ elements versus 0. Each call only removes one element, so the recursion goes $N - 1$ levels deep (visible in the increasing indentation of the output) and the work is $(N - 1) + (N - 2) + \dots + 1$, which is the $O(N^2)$ worst case described by $T(N) = T(N - 1) + O(N)$.

How to avoid it: Choose the pivot better, for example a random element or the median of the first, middle and last elements, and swap it into position high before partitioning. Shuffling the array first also works. Any of these makes balanced splits likely even on sorted input.

4.4 Test Cases and Screenshots

```
PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob4class.cpp -o prob4class } ; if ($?) { .\prob4class }
original: 34 7 23 32 5 62 14 19
pivot = 19: 34 7 23 32 5 62 14 19
i = 0, j = 1: 7 34 23 32 5 62 14 19
i = 1, j = 4: 7 5 23 32 34 62 14 19
i = 2, j = 6: 7 5 14 32 34 62 23 19
pivot swapped into place, i + 1 = 3: 7 5 14 19 34 62 23 32
final position of pivot: index 3
pivot = 14: 7 5 14
pivot = 5: 7 5
pivot = 32: 34 62 23 32
pivot = 62: 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 14

already sorted copy
original: 5 7 14 19 23 32 34 62
pivot = 62: 5 7 14 19 23 32 34 62
pivot swapped into place, i + 1 = 7: 5 7 14 19 23 32 34 62
final position of pivot: index 7
pivot = 34: 5 7 14 19 23 32 34
pivot = 32: 5 7 14 19 23 32
pivot = 23: 5 7 14 19 23
pivot = 19: 5 7 14 19
pivot = 14: 5 7 14
pivot = 7: 5 7
sorted: 5 7 14 19 23 32 34 62
comparisons: 28
```

Figure 5: Test Case for Quick Sort (on A and on the already sorted copy)

5 Problem 5: Merge Sort

Description: Merge Sort is a stable, divide-and-conquer algorithm that recursively cuts the input array into halves until single-element sub-arrays remain, then merges adjacent sorted sub-arrays.

Question: Implement Merge Sort using explicit index-based temporary array allocation without dynamic growth primitives like `push_back`.

5.1 Problem Statement

Input: Array `arr`, starting index `low`, ending index `high`.

Task: Calculate $\text{mid} = \lfloor (\text{low} + \text{high})/2 \rfloor$. Recursively sort `arr[low...mid]` and `arr[mid + 1...high]`. Merge both halves using a temporary buffer of size $(\text{high} - \text{low} + 1)$. Print each subarray when it is split and after it is merged, and count the comparisons made while merging (including those of the final merge).

Output: The mutated array `arr` sorted in ascending order, plus the comparison total and the number of comparisons in the final merge.

Constraints: $0 \leq \text{low} \leq \text{high} < N$.

Edge Cases:

- $\text{low} \geq \text{high}$: Base condition (1 element or empty range).
- Large input arrays: Guaranteed $O(N \log N)$ performance regardless of order.

5.2 Description of Approach

5.2.1 Algorithm Chosen and Correctness Explanation

The implementation uses index-assigned buffer merging:

1. **Divide Phase:** Compute `mid` and divide the problem into two recursive calls.
2. **Merge Phase:** Allocate temporary buffer `merged` of size $N_{\text{sub}} = \text{high} - \text{low} + 1$.
3. **Two-Pointer Selection:** Pointer i tracks the left half `[low...mid]`, j tracks `[mid + 1...high]`, and k tracks `merged`. Compare `arr[i]` and `arr[j]`, write the smaller value into `merged[k]`, and advance pointers.
4. **Copyback:** Copy remaining unconsumed elements, then write `merged` back to `arr[low...high]`.

Correctness Justification: By structural induction, the sub-arrays are sorted prior to merging. Two-pointer comparison selects the minimal uncopied element across both sub-arrays at every step, creating a merged array that is fully sorted.

5.2.2 Pseudocode

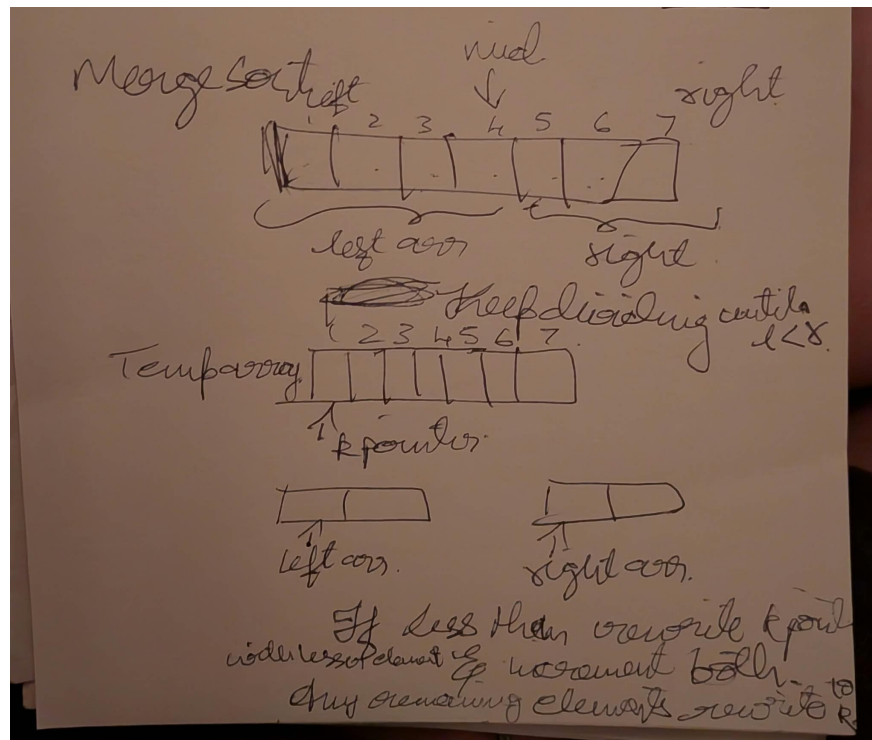


Figure 6: Flowchart illustrating the Merge Sort logic.

Algorithm 5 Merge Sort with Dynamic Allocation, Print Tracing, and Comparison Tracking

```

1: procedure MERGE(arr, low, mid, high, ref comparisons)
2:   merged  $\leftarrow$  new int array of size high – low + 1            $\triangleright$  temp array to store values
3:   i  $\leftarrow$  low
4:   j  $\leftarrow$  mid + 1
5:   k  $\leftarrow$  0
6:   while i  $\leq$  mid and j  $\leq$  high do
7:     comparisons  $\leftarrow$  comparisons + 1
8:     if arr[i]  $\leq$  arr[j] then
9:       merged[k]  $\leftarrow$  arr[i]
10:      i  $\leftarrow$  i + 1
11:      k  $\leftarrow$  k + 1
12:     else
13:       merged[k]  $\leftarrow$  arr[j]
14:       j  $\leftarrow$  j + 1
15:       k  $\leftarrow$  k + 1
16:     end if
17:   end while
18:   while i  $\leq$  mid do
19:     merged[k]  $\leftarrow$  arr[i]
20:     i  $\leftarrow$  i + 1
21:     k  $\leftarrow$  k + 1
22:   end while
23:   while j  $\leq$  high do
24:     merged[k]  $\leftarrow$  arr[j]
25:     j  $\leftarrow$  j + 1
26:     k  $\leftarrow$  k + 1
27:   end while
28:   for p = 0 to high – low do
29:     arr[low + p]  $\leftarrow$  merged[p]
30:   end for
31:   free merged
32: end procedure

33: procedure MERGESORT(arr, low, high, ref comparisons, ref finalMerge)
34:   if low < high then
35:     print "split: "
36:     PRINTARRAY(arr + low, high – low + 1)
37:     mid  $\leftarrow$   $\lfloor (\textit{low} + \textit{high}) / 2 \rfloor$             $\triangleright$  keep splitting until low  $\geq$  high
38:     MERGESORT(arr, low, mid, comparisons, finalMerge)
39:     MERGESORT(arr, mid + 1, high, comparisons, finalMerge)
40:     before  $\leftarrow$  comparisons
41:     MERGE(arr, low, mid, high, comparisons)
42:     finalMerge  $\leftarrow$  comparisons – before            $\triangleright$  the last merge to run is the top level one
43:     print "merged: "
44:     PRINTARRAY(arr + low, high – low + 1)
45:   end if
46: end procedure

```

5.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis:

1. **Recurrence Relation:** Dividing takes $O(1)$, merging takes $O(N)$ operations at each level.

$$T(N) = 2T(N/2) + cN$$

2. **Tree Expansion:** Tree depth is $\log_2 N$. Total work per level equals cN .

$$T(N) = cN \log_2 N + O(N)$$

- **Worst-Case Time Complexity** — $O(N \log N)$: $O(N \log N)$ bound across all inputs.
- **Best-Case Time Complexity** — $O(N \log N)$: Splitting and merging run unconditionally.
- **Average-Case Time Complexity** — $O(N \log N)$: Uniform time cost.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(N)$: Requires $O(N)$ space for the auxiliary **merged** array, plus $O(\log N)$ stack frames.

5.3 Answers to the Assignment Questions

Input	Total comparisons	Final merge
A (original)	17	7
Sorted copy	12	4
Reversed copy	12	4

For A, 7 of the 17 comparisons happen in the final merge of the two sorted halves $[7, 23, 32, 34]$ and $[5, 14, 19, 62]$.

Why the count changes so little: Merge Sort always splits into equal halves, whatever the input order, so its recursion has the same shape and the same $\log_2 N$ levels every time. Only the number of comparisons per merge changes, between the smaller half's size (when one half is entirely smaller) and size -1 (when the halves interleave). For $N = 8$ this gives 12 as the minimum ($\frac{N}{2} \log_2 N$, reached by both the sorted and reversed copies) and $N \log_2 N - N + 1 = 17$ as the maximum (reached by A). Bubble Sort, by contrast, goes from 7 to 28 comparisons and Quick Sort from 14 to 28, because their work depends on the input order.

Extra memory: Merge Sort needs $O(N)$ extra memory for the temporary **merged** array (at most N integers in the top-level merge), plus $O(\log N)$ for the recursion stack.

5.4 Test Cases and Screenshots

```
original: 34 7 23 32 5 62 14 19
split: 34 7 23 32 5 62 14 19
split: 34 7 23 32
split: 34 7
merged: 7 34
split: 23 32
merged: 23 32
merged: 7 23 32 34
split: 5 62 14 19
split: 5 62
merged: 5 62
split: 14 19
merged: 14 19
merged: 5 14 19 62
merged: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 17 (final merge: 7)

sorted copy
original: 5 7 14 19 23 32 34 62
split: 5 7 14 19 23 32 34 62
split: 5 7 14 19
split: 5 7
merged: 5 7
split: 14 19
merged: 14 19
merged: 5 7 14 19
split: 23 32 34 62
split: 23 32
merged: 23 32
split: 34 62
merged: 34 62
merged: 23 32 34 62
```

Figure 7: Test Case for Merge Sort, Part 1

```
merged: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 12 (final merge: 4)

reversed copy
original: 62 34 32 23 19 14 7 5
split: 62 34 32 23 19 14 7 5
split: 62 34 32 23
split: 62 34
merged: 34 62
split: 32 23
merged: 23 32
merged: 23 32 34 62
split: 19 14 7 5
split: 19 14
merged: 14 19
split: 7 5
merged: 5 7
merged: 5 7 14 19
merged: 5 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
comparisons: 12 (final merge: 4)
PS D:\c++ dsa stuff>
```

Figure 8: Test Case for Merge Sort, Part 2 (sorted and reversed copies)

6 Problem 6: Heap Sort

Description: Heap Sort transforms an array into a Max-Heap binary tree representation, repeatedly extracts the maximum root element to the end of the array, and restores the heap property using an iterative **SiftDown** procedure.

Question: Implement Heap Sort using an iterative **SiftDown** method without auxiliary container allocations.

6.1 Problem Statement

Input: Array A of size N .

Task: 1. Build a Max-Heap in-place by calling **SiftDown** on all non-leaf nodes from index $\lfloor N/2 \rfloor - 1$ down to 0. 2. For end from $N - 1$ down to 1, swap root $A[0]$ with $A[\text{end}]$, reduce heap size to end, and call **SiftDown**($A, 0, \text{end}$). After every pass, print the heap part and the sorted part separately, and count the swaps made inside **SiftDown**.

Output: The mutated array A sorted in non-decreasing order, plus the number of swaps in **SiftDown**.

Constraints: $N \geq 0$.

Edge Cases:

- $N \leq 1$: Immediate termination.
- Inputs with all equal elements: Heap construction completes with $O(N)$ effort.

6.2 Description of Approach

6.2.1 Algorithm Chosen and Correctness Explanation

The implementation uses a 0-indexed binary heap structure:

1. **Tree Indexing Rules:** For parent node index r :
 - Left Child: $\text{child} = 2r + 1$.
 - Right Child: $\text{child} + 1 = 2r + 2$.
2. **Iterative SiftDown:** While $2r + 1 < \text{size}$, locate the larger child. If $A[r] \geq A[\text{child}]$, break. Otherwise, swap $A[r]$ with $A[\text{child}]$, count the swap, and update $r = \text{child}$.
3. **Build-Heap Phase:** Run **SiftDown** on nodes $\lfloor N/2 \rfloor - 1 \dots 0$ in $O(N)$ total time.
4. **Extraction Phase:** Swap $A[0]$ with $A[\text{end}]$ and restore heap on range $0 \dots \text{end} - 1$. (This extraction swap is not counted; only swaps inside **SiftDown** are.)

Correctness Justification: Building the Max-Heap guarantees that $A[0]$ is the maximum element of the remaining heap. Extracting $A[0]$ to index end builds the sorted array backwards from right to left, while **SiftDown** maintains the Max-Heap invariant for remaining elements.

6.2.2 Pseudocode

Algorithm 6 Heap Sort with Swap Counter and Array Tracing

```

1: procedure SIFTDOWN(arr, root, size, ref swaps)
2:   while  $2 \cdot \text{root} + 1 < \text{size}$  do
3:      $\text{child} \leftarrow 2 \cdot \text{root} + 1$ 
4:     if  $\text{child} + 1 < \text{size}$  and  $\text{arr}[\text{child} + 1] > \text{arr}[\text{child}]$  then
5:        $\text{child} \leftarrow \text{child} + 1$ 
6:     end if
7:     if  $\text{arr}[\text{root}] \geq \text{arr}[\text{child}]$  then
8:       break
9:     end if
10:    SWAP( $\text{arr}[\text{root}]$ ,  $\text{arr}[\text{child}]$ )
11:     $\text{swaps} \leftarrow \text{swaps} + 1$  ▷ only swaps made inside siftDown are counted
12:     $\text{root} \leftarrow \text{child}$ 
13:  end while
14: end procedure

15: procedure HEAPSORT(arr, n, ref swaps)
16:   for  $i = \lfloor n/2 \rfloor - 1$  downto 0 do
17:     SIFTDOWN(arr, i, n, swaps)
18:   end for
19:   print "Heap: "
20:   PRINTARRAY(arr, n)
21:   for  $\text{end} = n - 1$  downto 1 do
22:     SWAP( $\text{arr}[0]$ ,  $\text{arr}[\text{end}]$ )
23:     SIFTDOWN(arr, 0, end, swaps)
24:     print "end = ", end, ": heap: "
25:     PRINTARRAY(arr, end)
26:     print "sorted: "
27:     PRINTARRAY(arr + end,  $n - \text{end}$ )
28:   end for
29: end procedure

```

6.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the number of elements.

1. **Build Max-Heap Cost:** Summing over node heights h :

$$T_{\text{build}}(N) = \sum_{h=0}^{\lfloor \log_2 N \rfloor} \left\lceil \frac{N}{2^{h+1}} \right\rceil O(h) = O\left(N \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(N)$$

2. **Extraction Phase Cost:** Performs $N - 1$ extractions. Each **SiftDown** takes $O(\log N)$ time:

$$T_{\text{extract}}(N) = \sum_{k=1}^{N-1} O(\log k) = O(N \log N)$$

Total Exact Cost Function $T(N)$:

$$T(N) = T_{\text{build}}(N) + T_{\text{extract}}(N) = O(N) + O(N \log N) = O(N \log N)$$

- **Worst-Case Time Complexity** — $O(N \log N)$: Strict upper bound for all inputs.

- **Best-Case Time Complexity** — $O(N \log N)$: Required extraction phase maintains $O(N \log N)$ bound.
- **Average-Case Time Complexity** — $O(N \log N)$: Optimal comparison-based sorting performance.

Space Complexity Derivation:

- **Auxiliary Memory** — $O(1)$: Iterative **SiftDown** avoids recursion stack allocation, achieving $O(1)$ auxiliary space.

6.3 Answers to the Assignment Questions

Results on A: the max-heap after the build phase is $[62, 32, 34, 19, 5, 23, 14, 7]$, and the total number of swaps inside **SiftDown** is **12**.

The heap as a binary tree: the children of index k are at $2k + 1$ and $2k + 2$.

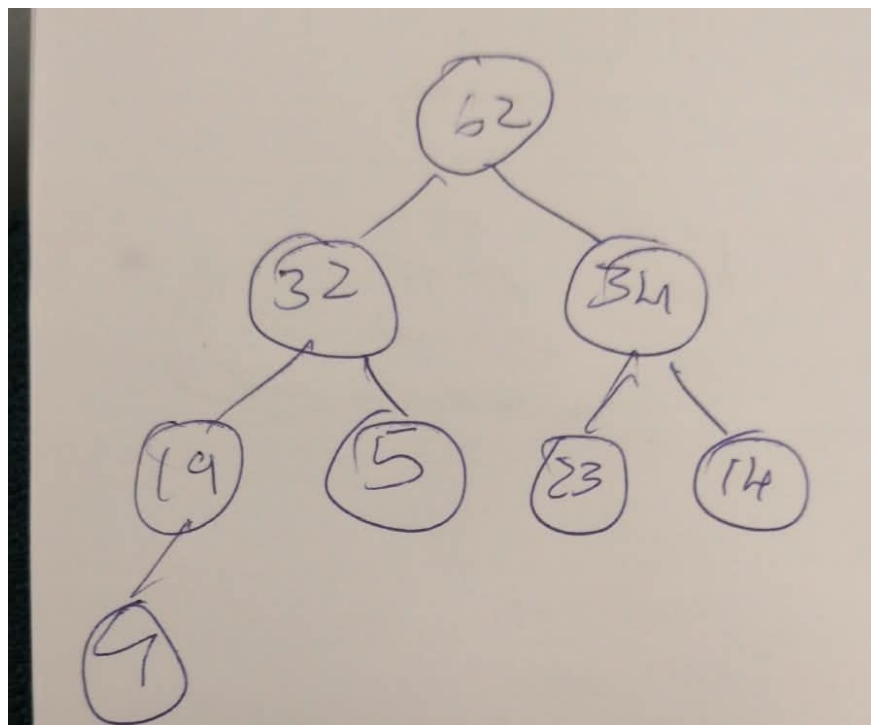


Figure 9: Max-heap $[62, 32, 34, 19, 5, 23, 14, 7]$ after the build phase.

Is Heap Sort stable? No. Label two equal values as 2_a and 2_b and take $[2_a, 2_b, 1]$. The build phase leaves it unchanged, since $2_a \geq 2_b$ and $2_a \geq 1$. The first extraction swaps the root with the last element, giving $[1, 2_b, 2_a]$, and then **SiftDown** on the root of size 2 swaps 1 with 2_b , giving $[2_b, 1, 2_a]$. The second extraction swaps the root with index 1, giving $[1, 2_b, 2_a]$. The final result has 2_b before 2_a , the reverse of their original order. The long-distance swaps between the root and the end of the array can move equal values past each other, so stability is not preserved.

6.4 Test Cases and Screenshots

```
PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob6class.cpp -o prob6class } ; if ($?) { .\prob6class }
original: 34 7 23 32 5 62 14 19
Heap: 62 32 34 19 5 23 14 7
end = 7: heap: 34 32 23 19 5 7 14
        sorted: 62
end = 6: heap: 32 19 23 14 5 7
        sorted: 34 62
end = 5: heap: 23 19 7 14 5
        sorted: 32 34 62
end = 4: heap: 19 14 7 5
        sorted: 23 32 34 62
end = 3: heap: 14 5 7
        sorted: 19 23 32 34 62
end = 2: heap: 7 5
        sorted: 14 19 23 32 34 62
end = 1: heap: 5
        sorted: 7 14 19 23 32 34 62
sorted: 5 7 14 19 23 32 34 62
swaps in siftDown: 12
PS D:\c++ dsa stuff>
```

Figure 10: Test Case for Heap Sort

7 Problem 7: Comparing the Six Algorithms

Description: Bubble Sort, Insertion Sort, and Selection Sort take $\Theta(n^2)$ time in the worst case, while Merge Sort and Heap Sort take $\Theta(n \log n)$. Quick Sort takes $\Theta(n \log n)$ on average but $\Theta(n^2)$ in its worst case. In this problem you will check these results with your own code.

Question: Write one C++ program that includes your sorting functions from Problems 1 to 6 (turn off the step-by-step printing) and your `printArray` function.

1. Make a table of the number of comparisons each algorithm makes on three inputs: A , a sorted copy of A , and a reversed copy of A .
2. Measure how long each algorithm takes to sort random arrays of 1,000, 5,000, and 10,000 integers between 1 and 100,000. Use `<chrono>` for timing and `<cstdlib>` (`rand`) or `<random>` for the values, and copy the same random data into a fresh array for every algorithm. Present the results as a table or a plot.
3. Call `printArray` on the first 10 elements of each sorted array to check that the sorting is correct.
4. Which algorithms grow like n^2 and which grow like $n \log n$? Do your measurements agree with the theory? Which algorithm would you choose for a large random array, and which for an array that is already almost sorted? Explain your choices.

7.1 Problem Statement

Input: Array $A = \{34, 7, 23, 32, 5, 62, 14, 19\}$ ($n = 8$), its pre-sorted copy $S = \{5, 7, 14, 19, 23, 32, 34, 62\}$, and its reverse-sorted copy $R = \{62, 34, 32, 23, 19, 14, 7, 5\}$. Additionally, dynamically generated random arrays of sizes $N \in \{1000, 5000, 10000\}$ with values bounded in $[1, 100000]$ produced via `std::mt19937`.

Task:

1. Instrument all six sorting routines with pass-by-reference comparison counters (`int &comparisons`) to count element-to-element comparisons on A , S , and R .
2. Benchmark wall-clock execution time (in milliseconds) across random arrays of size 1,000, 5,000, and 10,000 using `std::chrono::high_resolution_clock`.
3. Verify array correctness by printing the first 10 elements of each sorted array using `printArray`.
4. Analyze empirical runtime growth against theoretical bounds and justify optimal algorithm selection.

Output: Tabulated comparison counts ($n = 8$), execution runtimes (ms), and correctness verification output.

Constraints: $N \geq 0$; 32-bit signed integers; step-by-step intermediate output disabled during timing.

Edge Cases:

- $N = 0$ or $N = 1$: Trivial base cases (0 comparisons).
- Already sorted input (S): Best-case performance for Bubble Sort and Insertion Sort ($O(N)$ comparisons), worst case for Quick Sort with the last element as pivot.
- Reverse-sorted input (R): Worst-case quadratic comparisons for Bubble Sort, Insertion Sort and Quick Sort.

7.2 Description of Approach

7.2.1 Algorithm Chosen and Correctness Explanation

The driver program executes a multi-pass empirical evaluation:

1. **Comparison Counter Instrumentation:** Each sorting function receives an `int &comparisons` reference parameter initialized to 0 prior to invocation. Increments (`comparisons++`) are inserted directly before every element comparison.
2. **Data Isolation:** A single master random array `base` is generated for each size N . Before invoking each algorithm, `base` is copied into a fresh buffer `c` to guarantee identical input permutations.
3. **High-Resolution Timing:** Wall-clock time is measured using `std::chrono::high_resolution_clock` strictly surrounding the sorting function call, avoiding any allocation or print overhead.

Correctness Justification: Since each algorithm processes an identical copy of the unsorted input array, verifying that the first 10 elements match across all six algorithms confirms sorting correctness and consistency.

7.2.2 Pseudocode

Algorithm 7 Comparative Benchmarking Framework

```

1: procedure BENCHMARKSUITE
2:    $A \leftarrow [34, 7, 23, 32, 5, 62, 14, 19]$ 
3:    $S \leftarrow [5, 7, 14, 19, 23, 32, 34, 62]$ 
4:    $R \leftarrow [62, 34, 32, 23, 19, 14, 7, 5]$ 
                                      $\triangleright$  Part 1: Comparison Counting on Small Inputs
5:   for each algo in [Bubble, Insertion, Selection, Quick, Merge, Heap] do
6:     for each input in [ $A, S, R$ ] do
7:        $comparisons \leftarrow 0$ 
8:        $buffer \leftarrow \text{COPY}(input)$ 
9:       EXECUTEALGORITHM(algo, buffer, comparisons)
10:      RECORDCOMPARISONS(algo, input, comparisons)
11:    end for
12:  end for
                                      $\triangleright$  Parts 2 & 3: Timing and Verification on Random Arrays
13:   $sizes \leftarrow [1000, 5000, 10000]$ 
14:  for each  $N$  in  $sizes$  do
15:     $base \leftarrow \text{GENERATERANDOMARRAY}(N, 1, 100000)$ 
16:    for each algo in [Bubble, Insertion, Selection, Quick, Merge, Heap] do
17:       $buffer \leftarrow \text{COPY}(base)$ 
18:       $comparisons \leftarrow 0$ 
19:       $start \leftarrow \text{CHRONONOW}$ 
20:      EXECUTEALGORITHM(algo, buffer, comparisons)
21:       $end \leftarrow \text{CHRONONOW}$ 
22:       $elapsed \leftarrow \text{FORMATMILLISECONDS}(end - start)$ 
23:      PRINTARRAY(buffer, 10)
                                      $\triangleright$  Print first 10 sorted elements
24:    end for
25:  end for
26: end procedure

```

7.2.3 Time and Space Complexity Derivation

Step-by-Step Time Cost Analysis: Let N be the input size.

1. $\Theta(N^2)$ **Quadratic Algorithms (Bubble, Insertion, Selection):** Summing comparisons over N items yields quadratic equations:

$$T(N) = \sum_{i=1}^{N-1} i = \frac{N(N-1)}{2} = \Theta(N^2)$$

When N increases by a factor of 10 (e.g., $1,000 \rightarrow 10,000$), N^2 increases by a factor of $100\times$.

2. $\Theta(N \log N)$ **Linearithmic Algorithms (Merge, Heap, Quick Average):** Solving the recurrence $T(N) = 2T(N/2) + \Theta(N)$ via the Master Theorem yields:

$$T(N) = \Theta(N \log N)$$

When N increases from 1,000 to 10,000, $N \log_2 N$ increases by

$$\frac{10000 \cdot \log_2 10000}{1000 \cdot \log_2 1000} = 10 \cdot \frac{13.29}{9.97} \approx 13.3 \times .$$

Summary of Complexity Bounds:

Algorithm	Best Time	Average Time	Worst Time	Auxiliary Space
Bubble Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Insertion Sort	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$
Selection Sort	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$
Quick Sort	$O(N \log N)$	$O(N \log N)$	$O(N^2)$	$O(\log N)$
Merge Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$
Heap Sort	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$

Table 1: Theoretical Complexity Comparison across the Six Algorithms.

7.3 Empirical Results and Analysis

7.3.1 1. Comparison Counts ($n = 8$)

The exact comparison counts on the original (A), sorted (S), and reversed (R) arrays of size 8:

Algorithm	Array A (Original)	Sorted (S)	Reversed (R)
Bubble Sort	25	7	28
Insertion Sort	20	7	28
Selection Sort	28	28	28
Quick Sort	14	28	28
Merge Sort	17	12	12
Heap Sort	26	27	24

Table 2: Comparison Counts for $n = 8$ across Input Permutations.

7.3.2 2. Execution Time Benchmarks (in milliseconds)

Execution times measured on random arrays bounded in $[1, 100000]$:

Algorithm	N = 1,000	N = 5,000	N = 10,000
Bubble Sort	1.494 ms	40.329 ms	171.173 ms
Insertion Sort	0.562 ms	11.950 ms	50.212 ms
Selection Sort	0.811 ms	20.001 ms	79.008 ms
Quick Sort	0.070 ms	0.408 ms	0.839 ms
Merge Sort	0.174 ms	0.670 ms	1.834 ms
Heap Sort	0.093 ms	0.510 ms	1.283 ms

Table 3: Execution Runtime Benchmarks (ms) across Input Sizes (N).

7.3.3 3. Correctness Check

For every size, all six algorithms print the same first 10 elements, for example [53, 78, 507, 519, 553, 696, 707, 920, 1084, 1136] for $N = 1000$. Identical output from six different algorithms on the same input confirms that each one sorts correctly.

7.3.4 4. Theoretical vs. Empirical Growth Evaluation

- **Quadratic growth (n^2):** Bubble, Insertion and Selection Sort. From $N = 1,000$ to $N = 10,000$ the input grows $10\times$, and their runtimes grow by about $115\times$ (Bubble, $1.494 \rightarrow 171.173$ ms), $89\times$ (Insertion) and $97\times$ (Selection). The theory predicts $10^2 = 100\times$, so the measurements agree.
- **Linearithmic growth ($n \log n$):** Merge, Heap and Quick Sort (average case). Their runtimes grow by about $10.5\times$ (Merge), $13.8\times$ (Heap) and $12\times$ (Quick) over the same range. The theory predicts about $13.3\times$, so these also agree. Small deviations come from constant factors, caching and timer resolution, since these sorts take only about a millisecond.
- **Quick Sort worst case:** On the sorted input Quick Sort makes 28 comparisons, which is $\frac{N(N-1)}{2}$, so its $\Theta(n^2)$ worst case shows up when the pivot is the last element of a sorted array. The random arrays do not trigger it.

7.3.5 5. Algorithm Selection Justification

- **Choice for large random arrays: Quick Sort.** It was the fastest in the measurements (0.839 ms at $N = 10,000$), with small constant factors, good cache locality and $O(\log N)$ extra space. If a guaranteed $O(N \log N)$ worst case is needed, use **Merge Sort** (stable, but needs $O(N)$ extra memory) or **Heap Sort** (in-place, $O(1)$ extra memory).
- **Choice for nearly sorted arrays: Insertion Sort.** Its cost is $O(N + \text{inversions})$, which is close to linear when few elements are out of place (only 7 comparisons and 0 shifts on the sorted $n = 8$ input), and it needs no extra memory. Quick Sort with a last-element pivot should be avoided here, since it degrades to $O(N^2)$ on sorted or nearly sorted input.

7.4 Test Cases and Screenshots

```

PS D:\c++ dsa stuff> cd "d:\c++ dsa stuff\" ; if ($?) { g++ prob7class.cpp -o prob7class } ; if ($?) { .\prob7class }
comparisons (n = 8)
  algorithm      A      sorted  reversed
  bubble sort    25       7        28
  insertion sort 20       7        28
  selection sort 28       28       28
  quick sort     14       28       28
  merge sort     17       12       12
  heap sort      26       27       24

size 1000
  bubble sort    1.494 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136
  insertion sort 0.562 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136
  selection sort 0.811 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136
  quick sort     0.070 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136
  merge sort     0.174 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136
  heap sort      0.093 ms  first 10: 53 78 507 519 553 696 707 920 1084 1136

size 5000
  bubble sort    40.329 ms  first 10: 2 14 19 24 34 45 103 113 124 124
  insertion sort 11.950 ms  first 10: 2 14 19 24 34 45 103 113 124 124
  selection sort 20.001 ms  first 10: 2 14 19 24 34 45 103 113 124 124
  quick sort     0.408 ms  first 10: 2 14 19 24 34 45 103 113 124 124
  merge sort     0.670 ms  first 10: 2 14 19 24 34 45 103 113 124 124
  heap sort      0.510 ms  first 10: 2 14 19 24 34 45 103 113 124 124

size 10000
  bubble sort    171.173 ms  first 10: 4 19 23 25 41 60 63 66 72 75
  insertion sort 50.212 ms  first 10: 4 19 23 25 41 60 63 66 72 75
  selection sort 79.008 ms  first 10: 4 19 23 25 41 60 63 66 72 75
  quick sort     0.839 ms  first 10: 4 19 23 25 41 60 63 66 72 75
  merge sort     1.834 ms  first 10: 4 19 23 25 41 60 63 66 72 75
  heap sort      1.283 ms  first 10: 4 19 23 25 41 60 63 66 72 75

```

Figure 11: Test Case Execution Snapshot for Problem 7 showing comparison counts, runtime benchmarks, and first 10 sorted elements verification.